LAB 18 - PROMPTS

Diagnostics Stage: Automated Bias Checks before Go-Live

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab18-diagnose-bias/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Run the suspicious backtest (A9)

Do this (no prompt needed)
Run: python3 Lab18-diagnose-bias/buggy_backtest.py. Record the CAGR, Sharpe ratio and win rate.

Check: CAGR 71.52%, Sharpe 4.28, win rate 67.5% - far above anything a real SPY timing rule achieves.

Step 2: Find the problems (A9, K8)

Prompt - copy into your AI coding assistant
Review labs_fullcode/Lab18-diagnose-bias/buggy_backtest.py as a model-risk reviewer. For each line marked CHECK 1 to 3, explain the bias, how it inflates the result and the correct fix. Do not run or change the file.

Open in Claude | Open in ChatGPT

Check: CHECK 1 centred window (uses t+1 and t+2), CHECK 2 same-bar returns, CHECK 3 no costs.

Step 3: Build the look-ahead checks (A9, K8)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/diagnose.py. buggy_signal(bars) reproduces the exhibit (5-day rolling mean with center=True, long when it rises); trailing_signal(bars) is the fix (trailing window). In run(strategy=None, trials=None) add check 1, look-ahead in FEATURES, using pipeline.signals.lookahead_test (25 truncation truncation and future-shock checks) on the configured strategy and on the exhibit (fn=buggy_signal); and check 2, look-ahead in EXECUTION: the strategy's Sharpe with same-bar returns (target_t x return_t) versus the pipeline's next-bar backtest, plus the exhibit fixed one problem at a time: 0 as submitted (centred, same bar), 1 trailing window same bar, 2 + next-bar execution (common.backtest.backtest), 3 + alpaca_retail costs, and SPY buy and hold, as a CAGR / Sharpe / max drawdown table. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: The exhibit fails the truncation test; the configured strategy passes.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the cost, overfitting and survivorship checks (A9, K8)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/diagnose.py add: check 3 cost drag - gross vs net CAGR from cost_summary(), break-even from pipeline.backtest.break_even_bps versus the assumed one-way cost (half spread + slippage of the config preset): PASS if break-even >= 3 x assumed and drag < 25% of gross; check 4 overfitting - random_strategies(spy_returns, 500, seed=1) (skill-free long/flat signals switching with 10% probability a day, traded next bar), the best in-sample Sharpe among the first N where N = the number of sets in _outputs/Lab10_grid_results.csv (default 76), compared with the strategy's 2018-2021 Sharpe, and its 2022-2025 decay; check 5 survivorship / hindsight - the universe was chosen from today's listed symbols, and an equal-weight NVDA/MSFT/AAPL 'AI basket' versus SPY. Print each check as [PASS|WARN|FAIL|INFO] check subject evidence, a VERDICT ('BLOCK - fix the FAIL items' if the strategy has a FAIL, else 'CLEAR for paper trading - review every WARN'), write _outputs/Lab18_diagnostics.csv and charts Lab18_bias_comparison.png and Lab18_overfitting.png. main(argv): --strategy, --trials, --strict (exit 1 on BLOCK). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline diagnose --help shows --strategy, --trials and --strict.

Step 5: Run the diagnostics (A9)

Do this (no prompt needed)
Run: python -m pipeline diagnose (same as: python3 Lab18-diagnose-bias/diagnose_stage.py).

Check: ema_cross: look-ahead PASS (0 of 25), execution PASS (same-bar Sharpe 1.45 vs next-bar 0.82), cost drag PASS (gross 9.13% -> net 9.02%, break-even 135.8 bps vs 1.5 bps), overfitting WARN (IS Sharpe 0.86 vs best of 76 random 1.42; OOS 0.75), survivorship WARN. Exhibit: look-ahead FAIL (25 of 25), execution FAIL (4.28 -> 0.24). VERDICT for ema_cross: CLEAR for paper trading - review every WARN.

Step 6: Read the exhibit fixed one problem at a time (A9, K8)

Do this (no prompt needed)
Read the fix table and open _outputs/Lab18_bias_comparison.png.

Check: As submitted 71.47% / Sharpe 4.28; trailing window, same bar 66.56% / 4.68; + next-bar execution 2.23% / 0.24; + realistic costs 1.46% / 0.18; SPY buy and hold 14.14% / 0.78 - the same-bar error is the main bias.

Step 7: Read the overfitting and survivorship evidence (K8)

Do this (no prompt needed)
Read the INFO row and open _outputs/Lab18_overfitting.png. Then run python -m pipeline diagnose --strategy dip_buy.

Check: 500 random strategies: best IS Sharpe 1.77 -> OOS 0.55, median OOS 0.46, IS/OOS correlation -0.05, 16% have IS Sharpe > 1. AI basket CAGR 38.84% vs SPY 14.15%. dip_buy: cost drag PASS (break-even 19.5 bps), overfitting WARN (IS Sharpe 0.14, OOS 0.80).

Step 8: Use the stage as a gate and write the committee note (A9)

Prompt - copy into your AI coding assistant
Using labs_fullcode/_outputs/Lab18_diagnostics.csv, write a 4-sentence note for the investment committee: why the junior quant's Sharpe of 4.28 is not credible, which automated check caught each problem, what the configured ema_cross strategy's WARN items mean, and what must happen before any live capital.

Open in Claude | **Open in ChatGPT**

Check: Exit code 0 for ema_cross; a note that names look-ahead (features and execution), cost drag, overfitting and survivorship.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Run it

Step	Command / file
Run Lab 18	python3 Lab18-diagnose-bias/diagnose_stage.py (same as: python -m pipeline diagnose)
Also in this lab: buggy_backtest.py	python3 Lab18-diagnose-bias/buggy_backtest.py
The complete code	pipeline/diagnose.py (reference copy: Lab18-diagnose-bias/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.